

GenAI analysis - Assignment 2

Gen-AI # 1 - Google Gemini

1. I accessed this on my computer

2. My prompt:

Generate C code to implement the following requirements:

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

• Test File format:

Command Description

EMAIL <sender category> ,

<subject line> ,

<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

3. The given code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_CAPACITY 100

// Structure to represent an Email
typedef struct {
    char category[32];
    char subject[256];
    char date[16];
    int category_weight;
    long date_val; // YYYYMMDD for numerical comparison
} Email;
```

```

// Structure to represent the MaxHeap Priority Queue
typedef struct {
    Email* data;
    int size;
    int capacity;
} MaxHeap;

// Function declarations
MaxHeap* createHeap();
void freeHeap(MaxHeap* heap);
int getCategoryWeight(const char* category);
long parseDate(const char* date_str);
int compareEmails(Email* a, Email* b);
void swap(Email* a, Email* b);
void heapifyUp(MaxHeap* heap, int index);
void heapifyDown(MaxHeap* heap, int index);
void insertEmail(MaxHeap* heap, const char* category, const char* subject, const char* date);
Email* peekEmail(MaxHeap* heap);
void removeEmail(MaxHeap* heap);
void processInput(MaxHeap* heap);

int main() {
    MaxHeap* inboxHeap = createHeap();
    processInput(inboxHeap);
    freeHeap(inboxHeap);
    return 0;
}

// Create and initialize the MaxHeap
MaxHeap* createHeap() {
    MaxHeap* heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = INITIAL_CAPACITY;
    heap->size = 0;
    heap->data = (Email*)malloc(heap->capacity * sizeof(Email));
    return heap;
}

// Free allocated memory
void freeHeap(MaxHeap* heap) {
    if (heap) {
        free(heap->data);
        free(heap);
    }
}

// Assign category weight (Boss = highest, OtherPerson = lowest)
int getCategoryWeight(const char* category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0; // Default fallback
}

// Convert MM-DD-YYYY to a comparable integer YYYYMMDD
long parseDate(const char* date_str) {
    int mm, dd, yyyy;
    if (sscanf(date_str, "%d-%d-%d", &mm, &dd, &yyyy) == 3) {
        return (long)yyyy * 10000 + (long)mm * 100 + (long)dd;
    }
    return 0;
}

// Compare two emails: returns 1 if a has higher priority than b, 0 otherwise.
// Priority: Higher category weight first. If tie, newer date (larger date_val) first.
int compareEmails(Email* a, Email* b) {
    if (a->category_weight != b->category_weight) {
        return a->category_weight > b->category_weight;
    }
    return a->date_val > b->date_val;
}

void swap(Email* a, Email* b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

// Heapify up to maintain MaxHeap property after insertion
void heapifyUp(MaxHeap* heap, int index) {

```

```

while (index > 0) {
    int parent = (index - 1) / 2;
    if (compareEmails(&heap->data[index], &heap->data[parent])) {
        swap(&heap->data[index], &heap->data[parent]);
        index = parent;
    } else {
        break;
    }
}
}

// Heapify down to maintain MaxHeap property after deletion
void heapifyDown(MaxHeap* heap, int index) {
    int largest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < heap->size && compareEmails(&heap->data[left], &heap->data[largest])) {
        largest = left;
    }
    if (right < heap->size && compareEmails(&heap->data[right], &heap->data[largest])) {
        largest = right;
    }

    if (largest != index) {
        swap(&heap->data[index], &heap->data[largest]);
        heapifyDown(heap, largest);
    }
}

// Insert a new email into the MaxHeap
void insertEmail(MaxHeap* heap, const char* category, const char* subject, const char* date) {
    if (heap->size >= heap->capacity) {
        heap->capacity *= 2;
        heap->data = (Email*)realloc(heap->data, heap->capacity * sizeof(Email));
    }

    Email newEmail;
    strncpy(newEmail.category, category, sizeof(newEmail.category) - 1);
    newEmail.category[sizeof(newEmail.category) - 1] = '\0';

    strncpy(newEmail.subject, subject, sizeof(newEmail.subject) - 1);
    newEmail.subject[sizeof(newEmail.subject) - 1] = '\0';

    strncpy(newEmail.date, date, sizeof(newEmail.date) - 1);
    newEmail.date[sizeof(newEmail.date) - 1] = '\0';

    newEmail.category_weight = getCategoryWeight(newEmail.category);
    newEmail.date_val = parseDate(newEmail.date);

    heap->data[heap->size] = newEmail;
    heapifyUp(heap, heap->size);
    heap->size++;
}

// Peek at the highest priority email without removing it
Email* peekEmail(MaxHeap* heap) {
    if (heap->size == 0) return NULL;
    return &heap->data[0];
}

// Remove the highest priority email from the heap
void removeEmail(MaxHeap* heap) {
    if (heap->size == 0) return;
    heap->data[0] = heap->data[heap->size - 1];
    heap->size--;
    heapifyDown(heap, 0);
}

// Process command inputs from standard input
void processInput(MaxHeap* heap) {
    char line[512];

    while (fgets(line, sizeof(line), stdin)) {
        // Remove trailing newline character
        line[strcspn(line, "\r\n")] = 0;

        if (strncmp(line, "EMAIL", 5) == 0) {
            // Format: EMAIL <sender category>, <subject line>, <date>
            char *ptr = line + 5;
            while (*ptr == ' ') ptr++; // skip spaces

```

```

char category[32], subject[256], date[16];

// Parse comma-delimited fields
char *comma1 = strchr(ptr, ',');
if (!comma1) continue;
*comma1 = '\0';
strncpy(category, ptr, sizeof(category));

char *start_subject = comma1 + 1;
while (*start_subject == ' ') start_subject++; // trim leading space

char *comma2 = strchr(start_subject, ',');
if (!comma2) continue;
*comma2 = '\0';
strncpy(subject, start_subject, sizeof(subject));

char *start_date = comma2 + 1;
while (*start_date == ' ') start_date++; // trim leading space
strncpy(date, start_date, sizeof(date));

insertEmail(heap, category, subject, date);
}
else if (strcmp(line, "NEXT") == 0) {
    Email* top = peekEmail(heap);
    if (top != NULL) {
        printf("Sender: %s\n", top->category);
        printf("Subject: %s\n", top->subject);
        printf("Date: %s\n", top->date);
    }
}
else if (strcmp(line, "READ") == 0) {
    removeEmail(heap);
}
else if (strcmp(line, "COUNT") == 0) {
    printf("%d\n", heap->size);
}
}
}

```

Gen-AI # 2 Copilot

1. Accessed Copilot on my computer 2. Prompt:

Generate C code to implement the following requirements:

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

- Test File format:

Command Description
EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are

delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

• . You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

3. Given Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000

// ----- EMAIL STRUCT -----

typedef struct {
    char sender[50];
    char subject[200];
    char date[20];
    int priority;    // computed from sender category + date
    int yyyymmdd;    // numeric date for comparison
} Email;

// ----- HEAP STRUCT -----

typedef struct {
    Email heap[MAX_EMAILS];
    int size;
} MaxHeap;

// ----- PRIORITY RULES -----

int senderPriority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5;
    if (strcmp(sender, "Subordinate") == 0) return 4;
    if (strcmp(sender, "Peer") == 0) return 3;
    if (strcmp(sender, "ImportantPerson") == 0) return 2;
    return 1; // OtherPerson
}

int convertDate(const char *date) {
    // date format: MM-DD-YYYY
```

```

int mm, dd, yyyy;
sscanf(date, "%d-%d-%d", &mm, &dd, &yyyy);
return yyyy * 10000 + mm * 100 + dd;
}

```

```

// ----- HEAP FUNCTIONS -----

```

```

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

```

```

void heapifyUp(MaxHeap *h, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;

        // Compare priority first, then date
        if (h->heap[index].priority > h->heap[parent].priority ||
            (h->heap[index].priority == h->heap[parent].priority &&
             h->heap[index].yyyymmdd > h->heap[parent].yyyymmdd)) {
            swap(&h->heap[index], &h->heap[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

```

```

void heapifyDown(MaxHeap *h, int index) {
    while (1) {
        int left = index * 2 + 1;
        int right = index * 2 + 2;
        int largest = index;

        if (left < h->size) {
            if (h->heap[left].priority > h->heap[largest].priority ||
                (h->heap[left].priority == h->heap[largest].priority &&
                 h->heap[left].yyyymmdd > h->heap[largest].yyyymmdd)) {
                largest = left;
            }
        }

        if (right < h->size) {
            if (h->heap[right].priority > h->heap[largest].priority ||
                (h->heap[right].priority == h->heap[largest].priority &&
                 h->heap[right].yyyymmdd > h->heap[largest].yyyymmdd)) {
                largest = right;
            }
        }

        if (largest != index) {
            swap(&h->heap[index], &h->heap[largest]);
        }
    }
}

```

```

        index = largest;
    } else {
        break;
    }
}
}

```

```

void insertEmail(MaxHeap *h, Email e) {
    if (h->size >= MAX_EMAILS) {
        printf("Heap full, cannot insert.\n");
        return;
    }
    h->heap[h->size] = e;
    heapifyUp(h, h->size);
    h->size++;
}

```

```

Email* getMax(MaxHeap *h) {
    if (h->size == 0) return NULL;
    return &h->heap[0];
}

```

```

void deleteMax(MaxHeap *h) {
    if (h->size == 0) return;
    h->heap[0] = h->heap[h->size - 1];
    h->size--;
    heapifyDown(h, 0);
}

```

// ----- COMMAND PROCESSING -----

```

void processEmailCommand(MaxHeap *h, char *line) {
    // Format:
    // EMAIL <sender>, <subject>, <date>

    char *data = line + 6; // skip "EMAIL "
    char sender[50], subject[200], date[20];

    char *token = strtok(data, ",");
    strcpy(sender, token);

    token = strtok(NULL, ",");
    strcpy(subject, token);

    token = strtok(NULL, ",");
    strcpy(date, token);

    Email e;
    strcpy(e.sender, sender);
    strcpy(e.subject, subject);
    strcpy(e.date, date);
    e.priority = senderPriority(sender);
    e.yyymmdd = convertDate(date);
}

```

```

        insertEmail(h, e);
    }

void processNext(MaxHeap *h) {
    Email *e = getMax(h);
    if (!e) {
        printf("No emails.\n");
        return;
    }
    printf("Sender: %s\n", e->sender);
    printf("Subject: %s\n", e->subject);
    printf("Date: %s\n", e->date);
}

void processRead(MaxHeap *h) {
    if (h->size == 0) {
        printf("No emails to read.\n");
        return;
    }
    deleteMax(h);
}

void processCount(MaxHeap *h) {
    printf("%d\n", h->size);
}

// ----- MAIN -----

int main() {
    MaxHeap heap;
    heap.size = 0;

    char line[500];

    while (fgets(line, sizeof(line), stdin)) {
        if (strncmp(line, "EMAIL", 5) == 0) {
            processEmailCommand(&heap, line);
        }
        else if (strncmp(line, "NEXT", 4) == 0) {
            processNext(&heap);
        }
        else if (strncmp(line, "READ", 4) == 0) {
            processRead(&heap);
        }
        else if (strncmp(line, "COUNT", 5) == 0) {
            processCount(&heap);
        }
    }

    return 0;
}

```

Code Comparison:

1. Correctness

Both Gemini and Copilot use a list based MaxHeap and prioritize emails by sender category and then by newest date. Both correctly implement EMAIL, NEXT, READ, and COUNT, and both allow two NEXT commands to display the same email and two READ commands to remove emails without a NEXT. Gemini is slightly better because it uses a separate compareEmails() function, making the priority comparison the same. Gemini also uses strncpy() for string handling. Copilot uses strcpy() without checking input size. Copilot also limits the heap to 1,000 emails, which is not required by the assignment. Copilot prints messages when NEXT or READ is used with no emails, while Gemini does not, which could cause extra output if the grader expects nothing.

2. Execution Time

Both programs have the same overall time complexity. Inserting an email takes $O(\log n)$ because of heapifyUp(), and removing an email takes $O(\log n)$ because of heapifyDown(). NEXT and COUNT both take $O(1)$ because they access the heap root or stored size. Copilot uses iterative functions, while Gemini uses a recursive heapifyDown().

3. Space Complexity

Both programs use $O(n)$ space to store the emails. Gemini dynamically starts with space for 100 emails and doubles its capacity when needed. Copilot uses a fixed array for 1,000 emails, which is simpler but limits the number of emails and reserves space even when fewer emails are used. Gemini's recursive heapifyDown() can use $O(\log n)$ additional stack space while Copilot's iterative version uses $O(1)$ additional space. Gemini is more flexible but Copilot uses less extra space.

4. Maintainability

Gemini is more maintainable because its functions have clear purposes like compareEmails() which keeps the priority rules in one place. Copilot also has good organization and comments, but it repeats the priority comparison code in both heapifyUp() and heapifyDown(). Copilot's fixed 1,000-email limit makes it less flexible. Overall, Gemini is easier to modify and maintain.

Selection:

I choose the Gemini program because I think it does a better job meeting the requirements and is easier to work with. Both programs have the same main time complexity with $O(\log n)$ for adding and removing emails and $O(1)$ for NEXT and COUNT. Gemini can grow its heap when needed while Copilot is limited to 1,000 emails. Gemini also has a compareEmails() function that keeps the priority rules in one place. Gemini also uses strncpy() instead of strcpy() which makes the string handling safer. Because of these differences I would use the Gemini code.

Code Improvement:

Correctness:

I improved the input handling by deleting copied strings. I also moved `heap->size++` before `heapifyUp()` so the insertion process is clearer and the heap size is updated before the heap is adjusted.

Execution:

I changed `heapifyDown()` from a recursive function to an iterative loop. Both insertion and removal remain $O(\log n)$ but the loop avoids the extra function calls from recursion.

Space Complexity:

I kept the dynamic heap, which uses $O(n)$ space and can grow when more emails are added. This is more flexible than using a fixed-size array because the program is not limited to a set number of emails.

Maintainability:

I kept the `compareEmails()` function so all of the priority rules are handled in one place. I also kept the heap operations separated into functions which makes the code easier to read and change.